



Lecture 13 — Missing Values and Outliers

BTDS331 — Introduction to Data Science | Module 2: Data Collection and Preprocessing

Goal: Clean data by handling missing values and outliers to make it reliable for analysis.

This notebook is a **teaching companion** for Lecture 13. For every concept and Pandas method in the lecture, you get:

1. **What it is** — a one-line definition
2. **Why we use it** — the purpose / real-world reason
3. **Code** — a runnable example on our own practice dataset

◇ We use the lecture's own **"Student Performance"** dataset (Students A–H, with `Age`, `Study_Hours`, `Marks`) — it already contains missing values (`NaN`) and an outlier (`Age = 200`), exactly like the slides.

Previous Lecture Recap

- Data Cleaning improves data quality
- Pandas is used for data manipulation and cleaning
- DataFrame stores data in rows and columns
- `read_csv()` loads CSV datasets
- `head()` and `tail()` inspect data
- `info()` shows structure and data types
- `describe()` provides statistical summaries
- `isnull()` helps identify missing values
- `duplicated()` helps identify duplicate records

Today's Learning Objectives

1. Identify missing values in a dataset
2. Understand why data becomes missing
3. Detect missing values using Pandas
4. Remove or replace missing values

5. Understand outliers
6. Identify outliers using statistical methods
7. Use Pandas to handle common data-quality problems

```
In [1]: import pandas as pd

df = pd.read_csv("student_performance_practice.csv")
df
```

```
Out[1]:
```

	Student	Age	Study_Hours	Marks
0	A	20.0	3.0	65
1	B	21.0	4.0	72
2	C	NaN	5.0	78
3	D	20.0	6.0	84
4	E	22.0	NaN	88
5	F	21.0	5.0	91
6	G	200.0	4.0	75
7	H	20.0	3.0	95

1. What are Missing Values?

Definition: A **missing value** is a data value that is not available, not recorded, or not observed for a particular observation.

Common Representations: NaN , None , NA , <NA> , or simply a **blank** cell.

◇ **Key Point:** Missing data does **not** always mean the value is zero. Treating a missing Age as 0 (a newborn) or a missing Salary as 0 (unemployed) can silently corrupt your analysis — a missing value means "we don't know", not "it is zero".

```
In [2]: df.info()    # Non-Null Count column shows which fields have missing data
```

```

<class 'pandas.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Student         8 non-null     str
1   Age             7 non-null     float64
2   Study_Hours     7 non-null     float64
3   Marks           8 non-null     int64
dtypes: float64(2), int64(1), str(1)
memory usage: 388.0 bytes

```

2. Why Do Values Become Missing?

Common Causes:

- Data entry errors
- Sensor failure
- User skipped a field
- Survey non-response
- Data not collected
- Database integration problems
- Communication failure
- Privacy restrictions

Example: A weather sensor stops working for 2 hours → the Temperature readings for those two hours are recorded as NaN , even though the sensor was working fine before and after.

Why we care: The *reason* a value is missing often tells you the *right way* to handle it (see Section 3 — Types of Missing Data).

3. Types of Missing Data

Understanding *why* the data is missing (its "missingness mechanism") should always come **before** you decide how to fill or drop it.

Type	Meaning	Example
MCAR — Missing Completely At Random	The missingness has no relationship at all to the data (observed or unobserved)	Random blank cells scattered across a dataset due to a random glitch
MAR — Missing At	The missingness is related to other observed variables ,	Younger, less-educated respondents are more likely to skip the "Income" question

Type	Meaning	Example
Random	not the missing value itself	
MNAR — Missing Not At Random	The missingness is related to the missing value itself or an unobserved factor	People with a high salary are less likely to report it (so missing "Reported Salary" tends to correspond to high earners)

❖ **Practical Point:** Before filling missing values, understand **why** the data is missing. Blindly filling everything with the mean can hide a real pattern (e.g. in MNAR, the missing values are systematically different from the observed ones).

4. Detecting Missing Values with Pandas — `df.isna()` / `df.isnull()`

What it is: Returns a DataFrame of the same shape with `True` where a value is missing and `False` where a value is present. `isna()` and `isnull()` are **exact aliases** of each other — they do exactly the same thing, so you can use whichever reads more naturally.

Why we use it: To locate exactly *where* the missing values sit in the dataset, cell by cell.

```
df.isna()      # or
df.isnull()    # identical result
```

```
In [3]: df.isna()      # True = missing, False = present
```

```
Out[3]:
```

	Student	Age	Study_Hours	Marks
0	False	False	False	False
1	False	False	False	False
2	False	True	False	False
3	False	False	False	False
4	False	False	True	False
5	False	False	False	False
6	False	False	False	False
7	False	False	False	False

```
In [4]: df.isnull()    # exact same result as isna() – just a different name for
```

Out[4]:

	Student	Age	Study_Hours	Marks
0	False	False	False	False
1	False	False	False	False
2	False	True	False	False
3	False	False	False	False
4	False	False	True	False
5	False	False	False	False
6	False	False	False	False
7	False	False	False	False

5. Counting Missing Values — `df.isnull().sum()`

What it is: Chains `.sum()` onto the True/False grid from Section 4. Since `True` counts as `1` and `False` as `0`, summing down each column gives the **missing-value count per column**.

Why we use it: A quick, single-line health-check that tells you exactly which columns need attention, and how many rows are affected in each.

```
In [5]: df.isnull().sum()
```

```
Out[5]: Student      0
Age              1
Study_Hours      1
Marks            0
dtype: int64
```

Interpretation (matches the lecture example):

- `Age` → 1 missing value (Student C)
- `Study_Hours` → 1 missing value (Student E)
- `Marks` → 0 missing values
- Other columns → 0 missing values

6. Finding Rows That Contain Missing Values — `df[df.isnull().any(axis=1)]`

What it is: `df.isnull().any(axis=1)` checks **each row** and returns `True`

if **at least one** column in that row is missing (`axis=1` means "check across columns, per row"). Wrapping it in `df[...]` filters the DataFrame down to just those rows.

Why we use it: `isnull().sum()` tells you *how many* values are missing per column, but not which **actual records** are affected. This lets you inspect the real rows before deciding how to handle them.

```
In [6]: df[df.isnull().any(axis=1)]
```

```
Out[6]:
```

	Student	Age	Study_Hours	Marks
2	C	NaN	5.0	78
4	E	22.0	NaN	88

7. Handling Missing Values — Three Common Approaches

Once missing values are found, you have three broad options:

1. 💎 **Remove** rows/columns — `dropna()`
2. ➡ **Fill** with a suitable value (constant, mean, median, or a more advanced estimate/impute)
3. 💎 **Keep** the missing values as-is, when appropriate (e.g. the missingness itself is meaningful, or the column won't be used in analysis)

Let's go through each in detail.

7.1 Method 1 — Remove Missing Data: `df.dropna()`

What it is: `dropna()` removes any row (by default) that contains **at least one** missing value. Passing `axis=1` instead removes **columns** that contain any missing value.

Why we use it: The simplest fix — but only appropriate when the number of affected rows/columns is small, since it **permanently discards information**.

```
df_clean = df.dropna()           # remove ROWS with any missing value
df_clean = df.dropna(axis=1)     # remove COLUMNS with any missing value
```

⚠ **Use Carefully.** Removing too many records can:

- Reduce dataset size
- Remove useful information
- Introduce bias (if the missing rows aren't random — see MAR/MNAR above)

```
In [7]: # Remove ROWS that contain any missing value
df_dropna_rows = df.dropna()
print("Shape before:", df.shape)
print("Shape after: ", df_dropna_rows.shape)
df_dropna_rows
```

Shape before: (8, 4)
Shape after: (6, 4)

```
Out[7]:
```

	Student	Age	Study_Hours	Marks
0	A	20.0	3.0	65
1	B	21.0	4.0	72
3	D	20.0	6.0	84
5	F	21.0	5.0	91
6	G	200.0	4.0	75
7	H	20.0	3.0	95

```
In [8]: # Remove COLUMNS that contain any missing value
df_dropna_cols = df.dropna(axis=1)
print("Columns before:", df.columns.tolist())
print("Columns after: ", df_dropna_cols.columns.tolist())
df_dropna_cols
```

Columns before: ['Student', 'Age', 'Study_Hours', 'Marks']
Columns after: ['Student', 'Marks']

```
Out[8]:
```

	Student	Marks
0	A	65
1	B	72
2	C	78
3	D	84
4	E	88
5	F	91
6	G	75
7	H	95

7.2 Method 2 — Fill Missing Values:

`df["col"].fillna(...)`

What it is: `fillna()` replaces missing values in a column (or the whole DataFrame) with a value you provide — a constant, or a computed statistic like the mean or median.

Why we use it: Preserves the row instead of deleting it — useful when you can make a reasonable estimate of what the missing value probably was.

a) Fill with a constant

```
df["City"] = df["City"].fillna("Unknown")
```

Useful for categorical/text columns where "Unknown" or "Not Specified" is a meaningful label.

b) Fill numerical data with the mean

```
df["Marks"] = df["Marks"].fillna(df["Marks"].mean())
```

Good for roughly symmetric numeric data without extreme outliers.

c) Fill with the median

```
df["Age"] = df["Age"].fillna(df["Age"].median())
```

Better when the column is skewed or has outliers, since the median isn't pulled by extreme values (see Section 8 — Mean vs Median).

```
In [9]: # a) Fill a text/categorical column with a constant
df_fill = df.copy()
# (Our dataset has no text column with missing values, but this is how yo
example_city = pd.Series(["Bengaluru", None, "Delhi"])
example_city.fillna("Unknown")
```

```
Out[9]: 0    Bengaluru
        1      Unknown
        2      Delhi
        dtype: str
```

```
In [10... # b) Fill numerical data with the MEAN
df_fill["Study_Hours_mean_filled"] = df_fill["Study_Hours"].fillna(df_fil
df_fill[["Student", "Study_Hours", "Study_Hours_mean_filled"]]
```

Out[10]:

	Student	Study_Hours	Study_Hours_mean_filled
--	---------	-------------	-------------------------

0	A	3.0	3.000000
1	B	4.0	4.000000
2	C	5.0	5.000000
3	D	6.0	6.000000
4	E	NaN	4.285714
5	F	5.0	5.000000
6	G	4.0	4.000000
7	H	3.0	3.000000

```
In [11]: # c) Fill numerical data with the MEDIAN
df_fill["Age_median_filled"] = df_fill["Age"].fillna(df_fill["Age"].median())
df_fill[["Student", "Age", "Age_median_filled"]]
```

Out[11]:

	Student	Age	Age_median_filled
--	---------	-----	-------------------

0	A	20.0	20.0
1	B	21.0	21.0
2	C	NaN	21.0
3	D	20.0	20.0
4	E	22.0	22.0
5	F	21.0	21.0
6	G	200.0	200.0
7	H	20.0	20.0

⚠ Notice something important above: Student G's Age = 200 is an **outlier**, not missing — so it still pulls the **mean** upward if we ever computed one for Age. This is exactly why the lecture recommends **median** for Age here, and why outliers must be handled *before* (or together with) missing values, not after.

8. Mean vs Median — Which Should You Fill With?

Why this matters: The mean is **strongly affected** by extreme values, while the median is **far less affected**.

Example — suppose salaries are: 30,000 · 35,000 · 40,000 · 42,000 ·

500,000

- **Mean** = $(30,000 + 35,000 + 40,000 + 42,000 + 500,000) / 5 = \mathbf{129,400}$ ← pulled way up by 500,000
- **Median** = the middle value when sorted = **40,000** ← unaffected by the extreme value

Practical Rule:

- Use **Mean** → for relatively symmetric data (no big outliers)
- Use **Median** → for skewed data / data with extreme values

```
In [12... salaries = pd.Series([30000, 35000, 40000, 42000, 500000])
print("Mean: ", salaries.mean())
print("Median:", salaries.median())
```

Mean: 129400.0

Median: 40000.0

9. Real-World Example: Hospital Patient Data

Patient	Age	Blood Pressure
A	25	120
B	31	118
C	NaN	122
D	29	121
E	34	NaN

Possible approach:

- **Age** → fill with the **median age**
- **Blood Pressure** → **investigate why** the value is missing (was the reading skipped? did the device fail?) before deciding — this is a medical value and a wrong guess could be dangerous
- **Do not** blindly replace every missing value with zero (a Blood Pressure of 0 is not a real, safe value — it would look like a medical emergency!)

◇ **Key Principle:** The correct method depends on the **meaning** of the data — always ask what a missing value in this specific column really represents before choosing how to handle it.

10. What is an Outlier?

Definition: An **outlier** is an observation that is unusually far from the other observations in a dataset.

Example: 20, 21, 22, 23, 24, 25, 150 → 150 is an outlier — it sits far away from the rest of the cluster.

△ **Important:** An outlier is **not automatically an error**. It may represent:

- A measurement error
- A data entry error
- A rare (but real) event
- A genuine extreme observation

Real-World Outlier Examples

- Salary dataset: ₹30K, ₹35K, ₹32K, ₹40K, ₹38K, **₹500K** ← possible outlier
- Patient age = 250 (impossible — clearly an error)
- Temperature = 85°C (impossible for most contexts — likely a sensor fault)
- Transaction = ₹10 crore (could be genuine — a legitimate large purchase)
- Website visits = 10 million (could be genuine — a viral spike)

The core question is always: *Is it an error, or a genuine observation?*

11. Why Do Outliers Matter?

Outliers can **strongly affect**:

- Mean
- Standard deviation
- Correlation
- Regression models
- Statistical analysis in general

Example:

- Without outlier: 20, 21, 22, 23, 24 → a tight, symmetric distribution
- With outlier: 20, 21, 22, 23, 24, 200 → the distribution's mean and

spread **shift significantly**, even though only one value changed

Why we use it (in cleaning): If you don't detect and handle outliers, they can silently distort every downstream statistic and every model trained on the data.

```
In [13... no_outlier = pd.Series([20, 21, 22, 23, 24])
with_outlier = pd.Series([20, 21, 22, 23, 24, 200])

print("Without outlier -> mean:", no_outlier.mean(), " std:", round(no_outlier.mean(), 2))
print("With outlier -> mean:", with_outlier.mean(), " std:", round(with_outlier.mean(), 2))
```

```
Without outlier -> mean: 22.0 std: 1.58
With outlier -> mean: 51.666666666666664 std: 72.68
```

12. Detecting Outliers — Common Methods

1. **Domain Knowledge** — use real-world limits (e.g. human age can't be 250)
2. **Visualization** — box plots, scatter plots, histograms make outliers visually obvious
3. **IQR Method** — statistical rule based on quartiles (our focus for this course)
4. **Z-Score** — statistical rule based on standard deviations from the mean (basic interpretation, also covered below)

For this course, we focus on the IQR Method, with a basic interpretation of the Z-Score.

13. The IQR (Interquartile Range) Method

What it is: A statistical rule that flags any value falling far outside the "middle 50%" of the data as an outlier.

Steps:

1. Find **Q1** — the 25th percentile
2. Find **Q3** — the 75th percentile
3. Calculate **IQR** = Q3 – Q1

Outlier boundaries:

- **Lower Bound** = Q1 – 1.5 × IQR

- **Upper Bound** = $Q3 + 1.5 \times IQR$

Values outside these limits are commonly flagged as outliers.

Worked Example (from the lecture)

Data: 10, 12, 13, 14, 15, 16, 18, 20, 50

- Suppose $Q1 = 13$, $Q3 = 18$
- $IQR = 18 - 13 = 5$
- Upper Bound = $18 + (1.5 \times 5) = 25.5$
- Since $50 > 25.5 \rightarrow 50$ is flagged as an outlier

```
In [14... data = pd.Series([10, 12, 13, 14, 15, 16, 18, 20, 50])

Q1 = data.quantile(0.25)
Q3 = data.quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

print(f"Q1={Q1}, Q3={Q3}, IQR={IQR}")
print(f"Lower bound={lower}, Upper bound={upper}")

outliers = data[(data < lower) | (data > upper)]
print("Outliers found:", outliers.tolist())

Q1=13.0, Q3=18.0, IQR=5.0
Lower bound=5.5, Upper bound=25.5
Outliers found: [50]
```

14. Detecting Outliers with Pandas (on our own dataset)

What it is: The same 3-step IQR calculation applied directly to a DataFrame column using `.quantile()`.

Why we use it: This is the exact reusable pattern you'll apply to any numeric column in any dataset.

```
Q1 = df["Salary"].quantile(0.25)    # 25th percentile
Q3 = df["Salary"].quantile(0.75)    # 75th percentile
IQR = Q3 - Q1                       # Interquartile Range

lower = Q1 - 1.5 * IQR              # Lower bound
upper = Q3 + 1.5 * IQR              # Upper bound
```

```
outliers = df[(df["Salary"] < lower) | (df["Salary"] > upper)]
print(outliers)
```

Let's run this on the `Age` column of our Student Performance dataset, where Student G has `Age = 200`.

```
In [15... Q1 = df["Age"].quantile(0.25)
Q3 = df["Age"].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

print(f"Q1={Q1}, Q3={Q3}, IQR={IQR}")
print(f"Lower bound={lower}, Upper bound={upper}")

age_outliers = df[(df["Age"] < lower) | (df["Age"] > upper)]
age_outliers
```

```
Q1=20.0, Q3=21.5, IQR=1.5
Lower bound=17.75, Upper bound=23.75
```

```
Out[15]:
```

	Student	Age	Study_Hours	Marks
6	G	200.0	4.0	75

```
In [16... # Same check on Marks, for practice
Q1_m = df["Marks"].quantile(0.25)
Q3_m = df["Marks"].quantile(0.75)
IQR_m = Q3_m - Q1_m
lower_m = Q1_m - 1.5 * IQR_m
upper_m = Q3_m + 1.5 * IQR_m

print(f"Marks -> Q1={Q1_m}, Q3={Q3_m}, IQR={IQR_m}, Lower={lower_m}, Upper={upper_m}")
df[(df["Marks"] < lower_m) | (df["Marks"] > upper_m)] # empty = no outliers
```

```
Marks -> Q1=74.25, Q3=88.75, IQR=14.5, Lower=52.5, Upper=110.5
```

```
Out[16]:
```

	Student	Age	Study_Hours	Marks
--	---------	-----	-------------	-------

15. Z-Score — Basic Interpretation

What it is: The Z-score measures **how many standard deviations** a value is away from the mean: $Z = (x - \text{mean}) / \text{std}$. A common rule of thumb is to flag values with $|Z| > 3$ as outliers.

Why we use it: A quick alternative to IQR, especially useful when data is roughly normally-distributed (bell-shaped).

```
df["z_score"] = (df["Age"] - df["Age"].mean()) / df["Age"].std()
outliers = df[df["z_score"].abs() > 3]
```

```
In [17... z_scores = (df["Age"] - df["Age"].mean()) / df["Age"].std()
print(z_scores)

z_outliers = df[z_scores.abs() > 3]
z_outliers
```

```
0    -0.387777
1    -0.373024
2         NaN
3    -0.387777
4    -0.358272
5    -0.373024
6     2.267650
7    -0.387777
```

Name: Age, dtype: float64

```
Out[17]:
```

	Student	Age	Study_Hours	Marks
--	---------	-----	-------------	-------

16. What Should We Do With Outliers?

⚠ **Do NOT automatically delete them.**

Possible actions:

1. 🔍 **Verify** — check the original data source
2. ⇌ **Correct** — fix obvious data-entry errors (e.g. 200 was probably meant to be 20)
3. 🔍 **Remove** — only when justified (confirmed error, and removal won't bias the dataset)
4. 🔍 **Transform** — apply a suitable transformation (e.g. log transform) when appropriate
5. 🔍 **Keep** — if the value represents a genuine, real event

🔍 **Remember:** The right action depends on the **context** and the **meaning** of the data — exactly the same principle as for missing values.

```
In [18... # Example: CORRECT the obvious data-entry error for Student G (Age 200 ->
df_corrected = df.copy()
df_corrected.loc[df_corrected["Age"] > 120, "Age"] = 20 # domain-knowledge
df_corrected
```

Out[18]:

	Student	Age	Study_Hours	Marks
0	A	20.0	3.0	65
1	B	21.0	4.0	72
2	C	NaN	5.0	78
3	D	20.0	6.0	84
4	E	22.0	NaN	88
5	F	21.0	5.0	91
6	G	20.0	4.0	75
7	H	20.0	3.0	95

17. Missing Values vs Outliers — Side by Side

	Missing Values	Outliers
State	Value is unavailable	Value is available
Representation	NaN / None / NA	Has an unusually extreme value
Handling	Can be removed or imputed	Can be investigated or treated
Cause	Caused by collection problems	May be error or genuine
Detection code	isna() / isnull()	IQR / Z-score / visualization

18. Practical Data Cleaning Workflow

The full pipeline taught in this lecture:

```
Load Dataset → Inspect Data → Check Missing Values →  
Investigate Missing Data  
→ Remove / Fill / Keep → Detect Outliers → Investigate  
Outliers  
→ Correct / Remove / Keep → Clean Dataset → Ready for  
Analysis
```

Let's run through the **entire pipeline end-to-end** on our Student Performance dataset.

```
In [19... # STEP 1: Load Dataset  
df = pd.read_csv("student_performance_practice.csv")  
  
# STEP 2: Inspect Data  
print("Shape:", df.shape)  
df.info()
```

```

Shape: (8, 4)
<class 'pandas.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Student         8 non-null     str
1   Age              7 non-null     float64
2   Study_Hours      7 non-null     float64
3   Marks            8 non-null     int64
dtypes: float64(2), int64(1), str(1)
memory usage: 388.0 bytes

```

```

In [20... # STEP 3: Check Missing Values
print("Missing values per column:")
print(df.isnull().sum())

```

```

Missing values per column:
Student      0
Age          1
Study_Hours  1
Marks        0
dtype: int64

```

```

In [21... # STEP 4: Investigate Missing Data – see the actual affected rows
df[df.isnull().any(axis=1)]

```

```

Out[21]:
```

	Student	Age	Study_Hours	Marks
2	C	NaN	5.0	78
4	E	22.0	NaN	88

```

In [22... # STEP 5: Remove / Fill / Keep – decide per column
df_stage1 = df.copy()

# Study_Hours: numeric, roughly symmetric -> fill with mean
df_stage1["Study_Hours"] = df_stage1["Study_Hours"].fillna(df_stage1["Study_Hours"].mean())

# Age: has a known outlier (200) that would distort the mean -> fill with median
df_stage1["Age"] = df_stage1["Age"].fillna(df_stage1["Age"].median())

df_stage1

```

Out[22]:

	Student	Age	Study_Hours	Marks
0	A	20.0	3.000000	65
1	B	21.0	4.000000	72
2	C	21.0	5.000000	78
3	D	20.0	6.000000	84
4	E	22.0	4.285714	88
5	F	21.0	5.000000	91
6	G	200.0	4.000000	75
7	H	20.0	3.000000	95

In [23... *# STEP 6: Detect Outliers (IQR method) on Age*

```

Q1 = df_stage1["Age"].quantile(0.25)
Q3 = df_stage1["Age"].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = df_stage1[(df_stage1["Age"] < lower) | (df_stage1["Age"] > upper)]
print(f"Bounds: [{lower}, {upper}]")
outliers

```

Bounds: [18.125, 23.125]

Out[23]:

	Student	Age	Study_Hours	Marks
6	G	200.0	4.0	75

In [24... *# STEP 7: Investigate Outliers -> Student G's Age=200 is clearly a data-error (a human age of 200 is impossible)*

```

# STEP 8: Correct / Remove / Keep -> correct it (assume the '0' was dropped)
df_clean = df_stage1.copy()
df_clean.loc[df_clean["Age"] > 120, "Age"] = 20

df_clean

```

Out[24]:

	Student	Age	Study_Hours	Marks
0	A	20.0	3.000000	65
1	B	21.0	4.000000	72
2	C	21.0	5.000000	78
3	D	20.0	6.000000	84
4	E	22.0	4.285714	88
5	F	21.0	5.000000	91
6	G	20.0	4.000000	75
7	H	20.0	3.000000	95

```
In [25... # STEP 9: Clean Dataset -> Ready for Analysis. Final verification:
print("Missing values left:", df_clean.isnull().sum().sum())

Q1f = df_clean["Age"].quantile(0.25)
Q3f = df_clean["Age"].quantile(0.75)
IQRf = Q3f - Q1f
lowerf, upperf = Q1f - 1.5*IQRf, Q3f + 1.5*IQRf
print("Outliers left in Age:", ((df_clean['Age'] < lowerf) | (df_clean['A
df_clean.describe()
```

Missing values left: 0
Outliers left in Age: 0

Out[25]:

	Age	Study_Hours	Marks
count	8.000000	8.000000	8.000000
mean	20.625000	4.285714	81.000000
std	0.744024	1.030158	10.253919
min	20.000000	3.000000	65.000000
25%	20.000000	3.750000	74.250000
50%	20.500000	4.142857	81.000000
75%	21.000000	5.000000	88.750000
max	22.000000	6.000000	95.000000

19. Quick Reference Table

Method / Concept	What it does	When to use
<code>df.isna()</code> / <code>df.isnull()</code>	True/False grid of missing values	Locate missing cells
<code>df.isnull().sum()</code>	Missing value count per column	Quantify missing data

Method / Concept	What it does	When to use
<code>df[df.isnull().any(axis=1)]</code>	Rows with at least one missing value	Inspect affected records
<code>df.dropna()</code>	Remove rows with any missing value	Small number of affected rows
<code>df.dropna(axis=1)</code>	Remove columns with any missing value	Column is mostly missing / unusable
<code>df["col"].fillna(value)</code>	Fill missing values with a constant	Categorical/text column
<code>df["col"].fillna(df["col"].mean())</code>	Fill with the mean	Symmetric numeric data
<code>df["col"].fillna(df["col"].median())</code>	Fill with the median	Skewed data / has outliers
<code>Q1, Q3, IQR = quantile(.25), quantile(.75), Q3-Q1</code>	Interquartile Range	Outlier detection (main method)
<code>(x - mean)/std, flag \ Z\ > 3</code>	Z-score	Outlier detection (normal data)

20. Mini Practical Activity

Clean a Real Dataset — download any public dataset (e.g. from Kaggle) and perform:

1. Load the dataset using Pandas
2. Find missing values
3. Count missing values by column
4. Display rows containing missing values
5. Decide how to handle missing values (remove / fill / keep)
6. Identify numerical columns
7. Calculate Q1 and Q3
8. Calculate IQR
9. Identify potential outliers
10. Compare the dataset before and after cleaning

21. Key Takeaways

- Missing data is common in real-world datasets.
- Pandas provides `isna()`, `isnull()`, `dropna()`, and `fillna()`.
- Missing values should be **investigated** before treatment.
- An outlier is an unusually extreme observation.

- Outliers are **not always errors**.
- IQR is a practical method for detecting outliers.
- Domain knowledge is important before removing data.
- Clean data improves the reliability of analysis.

22. Practice Questions (from the lecture)

1. What is a missing value?
2. Give four reasons why data may be missing.
3. Differentiate MCAR, MAR, and MNAR.
4. How do you count missing values using Pandas?
5. Differentiate `dropna()` and `fillna()`.
6. What is an outlier?
7. Why should an outlier not automatically be deleted?
8. Explain the IQR method.
9. Write Pandas code to identify outliers using IQR.
10. Differentiate missing values and outliers.

Next Lecture: Data Formatting